Movie Recommendation System

This project implements the first stage of a **Movie Recommendation System**. The current version focuses on **movie input and user preferences**, which means the program is designed to collect and organize the data that will later be used by the recommendation algorithm and the data visualization stage.[cite:289]

Project goal

The purpose of this first deliverable is to let the user build a simple movie profile. The program allows users to manually add watched movies with ratings, import a shared movie list from a CSV file, save their favorite genres, actors, and directors, and view a summary of their movie history. This kind of structured user input is a common first step in recommendation systems because it creates the preference profile that later recommendation logic can use.[cite:113]

Current project structure

The code is intentionally split into small files so that each file has one clear responsibility. This makes the project easier to read, easier to maintain, and easier to extend when the group starts working on Deliverable 2 and Deliverable 3.[cite:289]

- `main.py`: runs the console application and handles the menu.
- `movie.py`: defines the `Movie` class.
- `preferences.py`: defines the `Preferences` class.
- `movie_service.py`: stores watched movies and provides helper methods.
- `data/movies.csv`: shared sample dataset used for CSV import.

What each file does

`movie.py`

This file defines the `Movie` class. Each object created from this class represents **one watched movie** and stores its title, release year, genre, rating, and optional information such as actor, director, and notes.

The class also defines a `__str__()` method so that each movie can be displayed in a clean and readable format when the user chooses to list watched movies.

`preferences.py`

This file defines the `Preferences` class. It stores the user's general tastes in three lists: favorite genres, favorite actors, and favorite directors.

The class includes setter methods to update each list. It also includes a `__str__()` method so the preferences can be printed in a simple and readable way from the console menu.

`movie_service.py`

This file contains the `MovieService` class, which acts as the main logic layer for movie management. Instead of storing and processing watched movies directly inside `main.py`, the program delegates that responsibility to this class, which helps keep the code more organized.[cite:289]

`MovieService` is responsible for:

- adding new movies,
- returning all watched movies,
- counting how many movies have been stored,
- calculating the average rating,
- finding the most watched genre,
- importing movies from a CSV file.

For CSV import, the program uses Python's built-in `csv.DictReader`, which reads a CSV file by using the header row as dictionary keys for each row.[cite:190] [cite:192]

`main.py`

This file is the entry point of the application. It creates the main menu, reads the user's choices, and calls the appropriate functions.

The menu currently supports these actions:

1. Add a watched movie manually.
2. Import watched movies from CSV.
3. Set user preferences.
4. Show watched movies.
5. Show preferences.
6. Show a summary.
7. Exit the program.

This file also contains the fixed shared path for the sample dataset:

```
DEFAULT_MOVIES_CSV_PATH = Path(__file__).parent / "data" / "movies.csv"
```

Using `Path(__file__)` is useful because it builds the CSV path relative to the project directory instead of depending on a hard-coded Windows path. This makes the project more portable for the whole group.[cite:249] [cite:294]

How the program works

When the program starts, `main.py` creates one `MovieService` object and one `Preferences` object. These objects stay in memory during execution and are updated whenever the user interacts with the menu.

Manual movie input

If the user selects the option to add a watched movie, the program asks for:

- title,
- year,
- genre,
- rating,
- optional actor,
- optional director,
- optional notes.

The program validates the input before creating a `Movie` object. For example, the title and genre cannot be empty, the year must be numeric, and the rating must be between 1 and 5.

CSV import

If the user selects the CSV import option, the program reads the shared file stored in `data/movies.csv`. Before importing, it checks whether the file exists. If the file is missing, the program shows a friendly error message instead of crashing.

The CSV file must include at least these columns:

- `title`
- `year`
- `genre`
- `rating`

Optional columns such as `actor`, `director`, and `notes` are also supported. Each valid row is converted into a `Movie` object and added to the watched movies list.[cite:190] [cite:195]

User preferences

The preferences menu lets the user input favorite genres, actors, and directors as comma-separated text. The program cleans that text by splitting it into items, removing extra spaces, and ignoring empty values.

These preferences are stored separately from the watched movies because they represent the user's general taste, not a single movie record. This distinction will be useful later when creating the recommendation algorithm.

Summary

The summary option gives a quick overview of the current user profile. At the moment, it shows:

- total number of watched movies,
- average rating,
- most watched genre.

This makes it easier to verify that the data has been stored correctly before moving on to the recommendation stage.

Why the CSV is stored inside the repository

The sample file `data/movies.csv` is stored inside the GitHub repository so that every team member can use the same dataset. Once the file is pushed to GitHub, any collaborator can download it with `git pull` and test the import option without manually creating their own dataset.[cite:261] [cite:275]

This approach also avoids hard-coded personal file paths such as `C:\Users\...`, which would only work on one computer. Keeping the CSV in the repository makes the project more consistent and easier to share across the team.[cite:249] [cite:251]

Why this design is useful for the next deliverables

This first deliverable does not yet recommend new movies. Its purpose is to build the input layer of the system: the watched movie history and the user preference profile.

This design is useful because the next stages can reuse the same data:

- Deliverable 2 can use watched movies and preferences as inputs for the recommendation algorithm.
- Deliverable 3 can use the same stored data to create charts, breakdowns by genre, and user trend analysis.

In other words, the current version provides the foundation on which the rest of the project will be built.[cite:113] [cite:289]

Notes for the team

- The virtual environment folder should **not** be uploaded to the repository.
- The `__pycache__` folder should also be ignored with `.gitignore`, because it contains automatically generated Python cache files rather than source code.
- The important files to keep in GitHub are the Python source files and the shared CSV data file.[cite:279] [cite:285]